Gamma Lab

Informe de rendimiento: línea base de Gamma Lab 1.0
Kernel, plugins, cómputo numérico y renderizado VTK – pensado para comparar contra Gamma Lab 2.0

Auditoría de rendimiento

16 de agosto de 2026

Resumen

Este informe documenta el rendimiento medido de **Gamma Lab 1.0** completo: arranque del kernel y del sistema de plugins, carga de archivos, corte de trials, preprocesamiento (filtro), cómputo numérico de cada plugin de análisis (FFT, PSD, Wavelet) y **todo** el renderizado VTK de la aplicación (los 19 archivos del repositorio que usan VTK). Ningún archivo de `core/` o `plugins/` fue modificado para producir este informe – todas las cifras provienen de ejecuciones reales de `pytest` contra el código real de la aplicación, con el archivo ABF real de `test/data/17308005.abf`, sin mockear ninguna fórmula matemática ni ninguna llamada a VTK. La suite que genera estos números vive en `test/performance_test/` y está explícitamente diseñada para volver a correrse, sin cambios de metodología, contra **Gamma Lab 2.0** durante su desarrollo este mes – la sección 16 explica el procedimiento exacto.

Índice

1. Resumen ejecutivo **2**

2. Objetivo y alcance **2**
2.1. Qué NO cubre este informe 2

3. Metodología **3**

4. Entorno de medición **3**

5. Inventario completo de VTK en Gamma Lab **5**
5.1. Patrón vectorizado (`numpy_to_vtk`) – 10 archivos 5
5.2. Patrón de bucle Python (`InsertNextValue` / `SetScalarComponentFromFloat`) – 5 archivos 5
5.3. Uso de VTK no relacionado con volumen de datos – 3 archivos 6

6. Resultados: kernel y sistema de plugins **7**

7. Resultados: carga de archivos y corte de trials **7**

8. Resultados: preprocesamiento (filtro) **7**

9. Resultados: cómputo de los plugins de análisis **8**

10. Resultados: renderizado VTK – patrón lento (bucle Python) **8**

11. Resultados: renderizado VTK – patrón vectorizado **9**
11.1. Los dos caminos ya vectorizados de `core/utils/adapters.py` 9
11.2. Micro-benchmark sintético: bucle vs. vectorizado 9

12. Estimaciones derivadas (no medidas en vivo) **10**
12.1. `filter_plugin._render_filtered` 10
12.2. Pipeline completo percibido por el usuario 10

13. Resultados: `DataStore` **10**

14. Hallazgos priorizados **11**

15. Recomendaciones para Gamma Lab 2.0 **11**

16. Metodología de comparación con Gamma Lab 2.0 **13**

17. Limitaciones conocidas **13**

18. Tabla completa de resultados (apéndice) **14**

19. Conclusión **15**

1 Resumen ejecutivo

Gamma Lab 1.0, medido en la máquina de referencia de la sección 4, tarda:

- **≈20 ms** en arrancar el kernel y cargar los 16 plugins (irrelevante frente al resto).
- **≈78 ms** en cargar un archivo ABF de 1.8 M muestras y cortarlo en 60 trials.
- **≈1.02 s** en total desde que el usuario pide un escalograma de wavelet de un solo trial hasta que aparece en pantalla – de los cuales el **95 %** son solo dos pasos: el CWT (216 ms) y el render del mapa 2D (725 ms).
- **≈12.6 s** (estimado por extrapolación lineal) en calcular el escalograma promedio de una sesión típica de 60 trials en `wavelet_average` – ya corre en un `QThread` y no congela la interfaz, pero sigue siendo el número más grande de toda la aplicación.

El hallazgo central es el mismo patrón, repetido en al menos cinco lugares del código: construir un arreglo VTK con un bucle de Python (`InsertNextValue`/`SetScalarComponentFromFloat` punto por punto) en vez de vectorizar con `numpy_to_vtk`, algo que **ya existe, ya funciona y ya se usa en 9 de los 19 archivos que usan VTK en el repositorio** (`core/utils/adapters.py` y sus consumidores). El micro-benchmark sintético de la sección 11.2 mide una diferencia de **65x a 150x** entre ambos patrones para el mismo volumen de datos. Vectorizar los cuatro puntos que faltan (`wavelet`, `wavelet_average`, `erp` para su mapa 2D, y `filter` para su gráfico de línea) es, con estos números sobre la mesa, la optimización de mayor impacto por esfuerzo disponible para Gamma Lab 2.0.

Cuello de botella	Costo medido	Hilo	Prioridad
`wavelet_average` – CWT ×60 trials	≈12.6 s *[estimado]*	QThread	**A VIGILAR**
`wavelet`/`wavelet_average` – render 2D	725–742 ms	Principal	**CUELLO DE BOTELLA**
`wavelet` – CWT (1 trial)	216 ms	Principal	**CUELLO DE BOTELLA**
`filter` – render de línea (señal completa)	≈336 ms *[estimado]*	Principal	**CUELLO DE BOTELLA**
`fft_average`/`psd_average` – cómputo	9–11 ms	Principal	**A VIGILAR**
Arranque del kernel y los 16 plugins	20 ms	Principal	**RÁPIDO**

2 Objetivo y alcance

Este informe tiene un único propósito: dejar una **línea base numérica y reproducible** del rendimiento de Gamma Lab 1.0 completo, para que dentro de este mismo mes – mientras se desarrolla Gamma Lab 2.0 – se pueda responder con datos, no con impresiones, a la pregunta "¿la versión nueva es más rápida?".

Por eso el alcance es deliberadamente amplio: no se midió solo el renderizado VTK (que ya se había medido parcialmente en `docs/reporte_tests.tex`, sección de rendimiento), sino **el kernel**, **el sistema de plugins**, **la carga de archivos**, **el corte de trials**, **el preprocesamiento** y **el cómputo de cada plugin de análisis**, además de repasar y ampliar la parte de VTK con un inventario completo de los 19 archivos del repositorio que lo usan (sección 5).

2.1 Qué NO cubre este informe

- **Corrección numérica** de los plugins (si el resultado matemático es el correcto) – eso ya está cubierto en `docs/reporte_tests.tex`, que valida cada plugin contra una referencia de MATLAB. Este informe asume que el código hace lo que dice hacer y mide únicamente cuanto tarda en hacerlo.
- La construcción de `MainWindow` (la ventana principal) – requiere una `QApplication` con un ciclo de eventos completo, no solo un stack VTK offscreen; ver sección 17.

- `FileIOService.load_edf`/`load_mat` – no hay ningún archivo `.edf`/`.mat` de referencia en `test/data/`.

3 Metodología

La suite completa vive en `test/performance_test/` y sigue exactamente la misma filosofía que el resto de la suite de pruebas del proyecto: **código real, datos reales, sin mocks**.

1. **Datos reales:** todos los benchmarks que necesitan una señal usan el mismo archivo ABF real, `test/data/17308005.abf` (2 canales, 1 800 000 muestras cada uno, 600 kHz→600 s a 3 kHz de tasa de muestreo), cortado en 60 trials con los mismos parámetros que `test/plugins_test/` (canal 0, canal de estímulo 1, umbral 0.7, ventana $t_0 = -0{,}05\,s$ a $t_1 = 4{,}00\,s$, modo `until_next_onset`).

2. **Código real, sin mockear:** cada benchmark llama directamente al método real del plugin o servicio bajo prueba (`plug._compute_fft`, `plug.compute_wavelet`, `plug.render_scalogram`, `FileIOService.load_abf`, `core.plugins.loader.discover`, etc.) – ninguna fórmula matemática ni ninguna llamada a VTK está sustituida por un doble (*mock*).

3. **Cronometraje:** `bench_common.time_block(fn, reps=N, warmup=W)` ejecuta la función `W` veces de calentamiento (se descartan, para no medir el costo de la primera importación/JIT/cache) y luego `N` veces cronometradas con `time.perf_counter()` (reloj monotónico de alta resolución). Se reporta mínimo, mediana, media y máximo; este informe usa siempre la **mediana** como cifra representativa, por ser más robusta a un outlier puntual que la media.

4. **Persistencia con etiqueta:** cada corrida completa de la suite se guarda en `results/perf_results.json` bajo una etiqueta (`GAMMA_PERF_LABEL`). Los números de este informe corresponden a la etiqueta `gammalab1_baseline_2026-08-16`, generada en la misma sesión en la que se escribió este documento.

5. **Snapshot de entorno:** cada corrida guarda automáticamente que SO, CPU, Python y versiones de librerías (VTK, numpy, scipy, Qt) se usaron (`bench_common.record_environment`), justamente para poder juzgar después si una diferencia de tiempos entre v1 y v2 es un cambio de código o un cambio de máquina.

4 Entorno de medición

Todas las cifras de este informe se midieron en una única corrida, en la siguiente máquina y con las siguientes versiones de librerías (capturado automáticamente por `bench_common.system_info()` y por `Get-CimInstance` de PowerShell):

Componente	Valor
CPU	13th Gen Intel(R) Core(TM) i7-13620H
Núcleos físicos / lógicos	10 / 16
RAM	31.7 GB
Sistema operativo	Windows 11 Home, build 26200 (64 bits)
Python	3.11.9 (entorno virtual del proyecto, `.venv`)
VTK	9.5.2
PyQt5 (Qt)	5.15.2
numpy	2.3.4
scipy	1.16.2
PyWavelets	1.8.0
pyabf	2.3.8

Por qué importa dejar esto por escrito: un número de rendimiento sin su entorno es casi inútil para comparar. Si Gamma Lab 2.0 se mide en otra máquina, con otra versión de VTK o de numpy, cualquier diferencia observada puede deberse al cambio de entorno y no al cambio de código. La sección 16 retoma este punto.

5 Inventario completo de VTK en Gamma Lab

Como este informe busca cubrir "todo el VTK de Gamma Lab", esta sección enumera los **19 archivos** del repositorio que importan `vtk`/`vtkmodules`, clasificados según el patrón que usan para convertir datos de NumPy a estructuras VTK.

5.1 Patrón vectorizado (`numpy_to_vtk`) – 10 archivos

Construyen el arreglo VTK de una sola vez con `vtk.util.numpy_support.numpy_to_vtk`, sin iterar en Python. Es el patrón rápido, medido en la sección 11.

- `core/utils/adapters.py` (el helper compartido: `dataset_to_vtk_table`, `trials_matrix_to_vtk_table`)
- `plugins/analysis/frequency/fft/`
- `plugins/analysis/frequency/fft_average/`
- `plugins/analysis/frequency/psd/`
- `plugins/analysis/frequency/psd_average/`
- `plugins/analysis/frequency/relative_psd/`
- `plugins/analysis/time/erp/` (solo su gráfico *butterfly*; el mapa 2D es aparte, ver abajo)
- `plugins/preprocessing/trials/`
- `plugins/io/open_signal/`
- `plugins/preprocessing/prepare/artifact_remove/` (usa `numpy_to_vtk` directo, sin pasar por `adapters.py`, pero el mismo patrón vectorizado)

5.2 Patrón de bucle Python (`InsertNextValue` / `SetScalarComponentFromFloat`) – 5 archivos

Construyen el arreglo VTK punto por punto, dentro de un `for` de Python. Es el patrón lento, medido en la sección 10 y estimado en la sección 12.

Archivo	Línea	Qué construye / estado
`wavelet_plugin.py`	334–336	Mapa 2D del escalograma. **Medido**: 725 ms para 3.05 M puntos.
`wavelet_average_plugin.py`	444–446	Mapa 2D del escalograma promedio. **Medido**: 742 ms para 3.05 M puntos.
`erp_plugin.py`	340–342	Mapa 2D *butterfly*. **Medido**: 36 ms – protegido por un tope interno (`MAX_SAMPLES=2000`) que recorta el eje de tiempo antes del bucle.
`filter_plugin.py`	251–254	Gráfico de línea de la señal filtrada. **Sin** tope de muestras – procesa la señal completa. **No medido en vivo** (requiere un `QVTKRenderWindowInteractor` real); **estimado** en ≈336 ms para 1.8 M muestras (§12).
`average_plugin.py`	110–112	Gráfico de línea del promedio entre trials. Protegido por el mismo tope `MAX_SAMPLES=2000` que `erp` – impacto estimado <0.5 ms, no es prioridad.

5.3 Uso de VTK no relacionado con volumen de datos – 3 archivos

Usan VTK pero no construyen arreglos de N puntos en un bucle, así que no aplica esta clasificación rápido/lento: `core/utils/vtk_context_menu.py` (menu contextual, interacción de usuario), `core/services/measurement_service.py` (mediciones interactivas click A/B) y `core/services/export_service.py` (exportar a imagen/tabla, un archivo por click). Ninguno de los tres aparece en este informe como benchmark porque su costo está dominado por I/O de disco o por interacción humana, no por cómputo.

6 Resultados: kernel y sistema de plugins

Mide `core/kernel.py` y `core/plugins/` reproduciendo exactamente la secuencia de arranque real de `main.py` (pasos 1–3, sin la ventana principal – ver sección 17): `PluginManager(plugins_dir).load_all()` (descubre + instancia los 16 plugins reales bajo `plugins/`) seguido de `kernel.register_plugin(meta.name, plugin)` para cada uno.

Benchmark	Mediana	
`plugins.discover`	19.6 ms	Escanea `plugins/`, valida `properties`
`PluginManager.load_all`	20.2 ms	`discover` + instanciar `PluginCls`
Arranque completo del kernel	19.9 ms	`load_all` + `kernel.register_plugin` ×16 (incluye `initi`
`Kernel.register_plugin` (aislado)	1.4 ms	Costo propio del kernel con 200 plugins *fi*

Interpretación: el arranque del kernel **no es un cuello de botella** – 20 ms es imperceptible comparado con cualquier otra operación de este informe (100× más rápido que un solo CWT de wavelet). Además, el propio `Kernel.register_plugin` aporta una fracción insignificante de ese costo (0.007 ms por plugin, medido de forma aislada con dobles *ficticios* que no importan PyQt5/VTK/scipy): casi todo el tiempo de arranque es el costo de (re)importar cada módulo de plugin real, que a su vez está dominado por la primera importación de PyQt5/VTK/scipy en el proceso (compartida entre los 16 plugins, no repetida). **Recomendación para 2.0:** no invertir esfuerzo de optimización aquí; el presupuesto de rendimiento rinde mucho más en las secciones siguientes.

7 Resultados: carga de archivos y corte de trials

Benchmark	Mediana	Tamaño
`FileIOService.load_abf`	59.8 ms	2 canales × 1 800 000 muestras
`trials.cut_trials_single_channel`	18.6 ms	salida 30 501 × 60

Juntas, cargar un archivo y generar sus trials toma ≈78 ms – el usuario lo percibe como prácticamente instantáneo. No se encontró ninguna operación cuadrática ni ningún recorrido redundante en ninguno de los dos caminos.

`FileIOService.load_edf` y `load_mat` no se midieron: `test/data/` no tiene ningún archivo `.edf`/`.mat` de referencia. Agregar uno (y su benchmark correspondiente en `test_io_bench.py`) es directo y queda como trabajo futuro.

8 Resultados: preprocesamiento (filtro)

`Filter_plugin.run_filter` (Butterworth pasa-banda, `scipy.signal.butter` + `sosfiltfilt`, orden 8, 0.5–4 Hz) es una función pura, benchmarkeada directamente:

Benchmark	Mediana	Tamaño
`run_filter` (señal completa)	43.2 ms	1 800 000 muestras
`run_filter` (1 trial)	1.8 ms	30 501 muestras

El cómputo del filtro en sí es rápido y escala linealmente con el tamaño, como se espera de `sosfiltfilt`. El costo real de .ªplicar Filtro.ªestá, en cambio, dominado por su **renderizado** – ver sección 12.

9 Resultados: cómputo de los plugins de análisis

Reproduce, con una corrida nueva en la misma máquina que el resto de este informe, la sección de rendimiento de `docs/reporte_tests.tex` (los números de esta sección reemplazan a los de esa auditoría anterior como referencia vigente). Salvo `wavelet_average`, todos corren hoy en el **hilo principal de Qt** sin `QThread` – la mediana es una medida directa de cuanto se congela la interfaz por cada clic en "Calcular".

Benchmark	Mediana	Hilo hoy
`fft._compute_fft` (60 trials)	3.8 ms	Principal
`fft_average._compute_fft_average`	11.5 ms	Principal
`psd._compute_psd` (Welch)	5.3 ms	Principal
`psd_average._compute_psd` (Welch)	9.2 ms	Principal
`relative_psd._compute_psd` (Welch)	6.3 ms	Principal
`relative_psd._compute_relative_psd` (razón)	0.07 ms	Principal
`wavelet.compute_wavelet` (1 trial, CWT)	**215.6 ms**	Principal
`wavelet_average.compute_wavelet` (×20 trials)	**4190.2 ms**	*QThread ya*

Lectura: FFT, PSD y PSD promedio son todos submilisegundo a un dígito de milisegundos – ninguno justifica moverse a un `QThread` por sí solo. `wavelet.compute_wavelet`, en cambio, es **60 veces más lento que el siguiente peor** (`fft_average`) y corre en el hilo principal: cada clic en "Calcular Wavelet" congela la interfaz ≈216 ms de forma perceptible. `wavelet_average` escala lo mismo por trial (≈209.5 ms/trial, consistente con el número de `wavelet` individual) pero ya corre en background – es el número más grande de todo el informe en términos absolutos (≈4.2 s para 20 trials, ≈12.6 s extrapolado a 60), pero al no bloquear la UI su prioridad de arreglo es menor que la de `wavelet.compute_wavelet`, que sí la bloquea.

10 Resultados: renderizado VTK – patrón lento (bucle Python)

Benchmark	Tamaño	Mediana
`wavelet.render_scalogram`	998×3051 (3.05 M puntos)	**725.3 ms**
`wavelet_average.render_scalogram`	998×3051 (3.05 M puntos)	**741.6 ms**
`erp._render_heatmap`	60×30501 → recortado a 60×1907	35.8 ms

Los tres construyen su `vtkImageData` con `img.SetScalarComponentFromFloat(i, j, 0, 0, valor)` dentro de un doble `for` de Python. `erp` sale barato en la comparación no porque su código sea distinto, sino porque un límite interno (`MAX_SAMPLES=2000`) ya recorta el eje de tiempo *antes* de entrar al bucle – en la práctica itera sobre ≈114 000 puntos, no 1.8 millones. `wavelet` y `wavelet_average` no tienen ese límite y pagan el costo completo de 3.05 M puntos.

Nota de variabilidad medida: en una corrida anterior de esta misma suite (14 de agosto de 2026, ver `results/perf_results.json`, etiqueta `v1_baseline`) este mismo benchmark de `wavelet.render_scalogram` midió 5.70 s – casi 8× más lento que en la corrida de este informe (725 ms), sin ningún cambio de código entre ambas. Es una variación atribuible al primer render offscreen de VTK en un proceso nuevo (inicialización de contexto gráfico), no al código. Al comparar Gamma Lab 1.0 contra 2.0 (§16) conviene correr cada lado 2–3 veces y mirar consistencia, no una única corrida.

11 Resultados: renderizado VTK – patrón vectorizado

11.1 Los dos caminos ya vectorizados de `core/utils/adapters.py`

Benchmark	Tamaño	Mediana
`adapters.dataset_to_vtk_table`	1 800 000 filas × 3 col.	**21.0 ms**
`adapters.trials_matrix_to_vtk_table`	30 501 filas × 61 col.	**11.1 ms**

Estos dos caminos alimentan los gráficos de línea de 9 de los 16 plugins (sección 5) y son, en volumen de datos equivalente o mayor, **órdenes de magnitud más rápidos** que el patrón de bucle: convertir la señal completa de 1.8 M muestras a una tabla VTK (21 ms) es más rápido que renderizar un solo escalograma de 3.05 M puntos con el patrón de bucle (725 ms) – pese a que el volumen de datos es comparable.

11.2 Micro-benchmark sintético: bucle vs. vectorizado

Para aislar el costo del patrón en sí (sin CWT, sin ABF, solo la conversión a VTK), se construyó el mismo arreglo de N números aleatorios con `vtkFloatArray.InsertNextValue` en un bucle de Python, y con `numpy_to_vtk`:

N puntos	Bucle Python	Vectorizado	Factor
50 000	4.58 ms	0.031 ms	**147.7x**
300 000	28.45 ms	0.432 ms	**65.8x**

El factor no es una constante exacta (el camino vectorizado tiene un costo fijo pequeño que pesa más a N chico, y a submilisegundo el ruido de medición es mayor), pero el orden de magnitud es consistente: **vectorizar cuesta entre 65 y 150 veces menos** que el bucle de Python, para el mismo volumen de datos, en esta máquina. Esta cifra es la que sostiene la estimación de la sección siguiente y la recomendación principal de este informe.

12 Estimaciones derivadas (no medidas en vivo)

Esta sección combina mediciones reales de otras secciones para estimar el costo de dos operaciones que no se pudieron benchmarkear en vivo por requerir una `QApplication` completa (ver sección 17). Se marcan explícitamente *[estimado]* y se muestra el cálculo, para que puedan reemplazarse por una medición real apenas exista un arnés de prueba adecuado.

12.1 `filter_plugin._render_filtered`

`filter_plugin.py:251-254` construye el gráfico de la señal filtrada con un `for` que llama dos veces a `InsertNextValue` por muestra (tiempo y señal), **sin** ningún tope de downsampling previo (a diferencia de `erp`/`average`, que recortan a 2000 puntos antes de renderizar). `on_apply_filter` llama a este render **dos veces** por clic: una para la señal completa y otra para el trial.

Tomando el costo medido por punto del micro-benchmark (§11.2: ≈93 ns por llamada a `InsertNextValue` sobre un arreglo, promediado entre los dos tamaños medidos) y contando 2 llamadas por iteración:

Render	Tamaño	Estimado
Señal completa	1 800 000 muestras	≈336 ms *[estimado]*
Trial individual	30 501 muestras	≈5.7 ms *[estimado]*
Total por clic en .ªplicar Filtro" (render) + (cómputo, §6, medido: 44.9 ms)		**≈386 ms** *[estimado]*

Si esta estimación es siquiera aproximadamente correcta, `filter` tiene el mismo problema que `wavelet`: un render sin vectorizar que domina por completo el tiempo total de la operación (87 % del estimado), muy por encima del costo real del filtro Butterworth en sí (13 %, medido). Es, junto con `wavelet`/`wavelet_average`, el otro candidato prioritario a vectorizar.

12.2 Pipeline completo percibido por el usuario

Sumando benchmarks medidos de distintas secciones (todos en el hilo principal, por lo que se perciben como un solo bloqueo de la interfaz):

Flujo	Tiempo total
Cargar ABF + cortar trials	78.4 ms (medido)
+ Ver escalograma wavelet de 1 trial	+941 ms (216 ms CWT + 725 ms render, medidos)
Total: abrir archivo → ver 1 escalograma	**≈1.02 s**

13 Resultados: `DataStore`

Benchmark	Mediana	Detalle
`add_signal` (×500)	13.0 ms	0.026 ms/señal; genera claves `raw_signal_N` con un `while` que revisa c
`set`+`get` (×2000 pares)	0.6 ms	0.00015 ms/operación – diccionario puro, O(1) como se espera.

`DataStore` no es, ni de cerca, un cuello de botella incluso en escenarios con muchas más señales cargadas de las que un usuario típico acumularia en una sesión. No requiere atención en 2.0.

14 Hallazgos priorizados

1. **CUELLO DE BOTELLA** **Renderizado 2D sin vectorizar en `wavelet/wavelet_average`** (725–742 ms, hilo principal) – el bloqueo de interfaz más grande y más fácil de eliminar de todo el informe: reemplazar el bucle `SetScalarComponentFromFloat` por `numpy_to_vtk` (mismo patrón que ya usa `core/utils/adapters.py`) puede reducirlo, según el micro-benchmark de §11.2, a pocos milisegundos.

2. **CUELLO DE BOTELLA** **`wavelet.compute_wavelet` en el hilo principal** (216 ms) – el segundo bloqueo más grande. A diferencia del render, este es un problema algorítmico/de threading, no solo de vectorización: mover el CWT a un `QThread` (como ya hace `wavelet_average`) resuelve el bloqueo de UI sin tocar la matemática.

3. **CUELLO DE BOTELLA** **Renderizado de línea sin vectorizar ni topar en `filter`** (≈336 ms estimado, hilo principal, sin tope de muestras) – mismo patrón que (1), pero además sin el límite de seguridad (`MAX_SAMPLES`) que sí tienen `erp`/`average`.

4. **A VIGILAR** `wavelet_average.compute_wavelet` (≈12.6 s estimado para 60 trials) – el número absoluto más grande del informe, pero de menor prioridad relativa porque ya corre fuera del hilo principal (QThread) y no congela la interfaz; sigue siendo candidato a paralelizar entre trials o cachear resultados parciales en 2.0.

5. **A VIGILAR** **Inconsistencia de patrón en `average_plugin.py`** (bucle sin vectorizar, pero topado a 2000 puntos, impacto <0.5 ms) – no es una prioridad de rendimiento, pero es la misma deuda de código que (1) y (3): en vez de 5 implementaciones distintas del mismo patrón, Gamma Lab 2.0 debería tener un único helper vectorizado para *cualquier* conversión NumPy→VTK (imagen 2D o tabla), reutilizado por todos los plugins.

6. **RÁPIDO** **Arranque del kernel, carga de ABF, corte de trials, `DataStore`, FFT/PSD** – todos por debajo de 80 ms, ninguno requiere optimización en 2.0. No gastar presupuesto de ingeniería aquí.

15 Recomendaciones para Gamma Lab 2.0

1. **Un único helper vectorizado para imágenes 2D**, análogo a `core/utils/adapters.py` pero para `vtkImageData`: recibir una matriz `Z` (frecuencia × tiempo, o trial × tiempo) y construir el `vtkImageData` con `numpy_to_vtk(Z.ravel(order="C")) + img.GetPointData().SetScalars(` en vez de que cada uno de `wavelet`, `wavelet_average` y `erp` reimplemente su propio doble `for`. Esto ataca los hallazgos (1) y (5) a la vez.

2. **Usar el helper de tablas ya existente en `filter_plugin.py`** en vez de su bucle propio de `InsertNextValue` – ataca el hallazgo (3), y de paso agregar el mismo tope `MAX_SAMPLES` que ya tienen `erp`/`average` como red de seguridad adicional.

3. **Mover todo cómputo pesado a `QThread` de forma sistemática**, no solo donde ya se hizo (`wavelet_average`): en particular `wavelet.compute_wavelet` (216 ms, hallazgo 2). El resto (FFT/PSD, <12 ms) no lo necesita hoy, pero conviene decidir un criterio único (p. ej. "todo cómputo >50 ms va a QThread") en vez de que sea caso por caso.

4. **Conservar esta suite de benchmarks como guardia de regresión**, corriéndola en cada cambio significativo de rendimiento durante el desarrollo de 2.0 – no solo al final. Ver sección 16.

5. **Evaluar cachear o paralelizar el CWT entre trials** en `wavelet_average` (hallazgo 4) – a 209.5 ms/trial, 60 trials en serie son ≈12.6 s; con 16 núcleos lógicos disponibles en la máquina de referencia, paralelizar los CWT independientes entre trials (son embarrasingly parallel) podría reducir ese número de forma significativa sin cambiar el algoritmo.

6. **Agregar un archivo `.edf` y/o `.mat` de referencia a `test/data/`** para poder benchmarkear (y validar contra MATLAB) los otros dos formatos que soporta `FileIOService`, hoy sin cobertura de ningún tipo.

16 Metodología de comparación con Gamma Lab 2.0

Esta es la parte pensada explícitamente para dentro de un mes. El procedimiento:

1. **No tocar** `bench_common.py` **ni** `compare_report.py` – son agnósticos a la versión de la app: solo cronometran funciones y comparan dos etiquetas del mismo JSON.
2. **Copiar (o adaptar) los archivos** `test_*_bench.py` al repositorio de Gamma Lab 2.0. Si 2.0 mantiene los mismos nombres de módulo/función (`FileIOService.load_abf`, `cut_trials_single_channel`, `_compute_fft`, `compute_wavelet`, `render_scalogram`, `core.plugins.loader.d` etc.), los archivos deberían correr prácticamente sin cambios. Si 2.0 renombra o reorganiza módulos, el único ajuste necesario son los `import` de cada archivo – la lógica de medición no cambia.
3. **Correr contra 1.0 con una etiqueta clara** (ya hecho en este informe):

   ```
   $env:GAMMA_PERF_LABEL = "gammalab1_baseline_2026-08-16"
   pytest test/performance_test -s
   ```

4. **Correr la misma suite contra 2.0**, en la **misma máquina** si es posible (o dejando registrado el entorno de cada lado, capturado automáticamente por `record_environment()` – ver §4):

   ```
   $env:GAMMA_PERF_LABEL = "gammalab2_baseline_2026-09-XX"
   pytest test/performance_test -s
   ```

5. **Comparar con el script ya existente:**

   ```
   python test/performance_test/compare_report.py ‘
       gammalab1_baseline_2026-08-16 gammalab2_baseline_2026-09-XX
   ```

   Esto imprime una tabla con el tiempo “antes”, “después” y el factor de aceleración por cada benchmark que exista en ambas etiquetas, además del entorno de cada lado.
6. **Repetir cada lado 2–3 veces** antes de sacar conclusiones sobre los benchmarks de renderizado VTK, por la variabilidad de primer render offscreen documentada en §10.

Qué preguntas debería poder responder esta comparación en 2.0:

- ¿Se corrigió el patrón de renderizado sin vectorizar? (factor esperado: 65–150x en los benchmarks de `wavelet`/`wavelet_average`/`filter`).
- ¿`wavelet.compute_wavelet` sigue bloqueando el hilo principal, o se movió a background?
- ¿El arranque del kernel se mantuvo liviano, o una arquitectura nueva (más plugins, más servicios) lo volvió perceptible?
- ¿La carga de archivos y el corte de trials se mantienen submedio-segundo?

17 Limitaciones conocidas

- `MainWindow` **no se benchmarkeó.** Requiere una `QApplication` real con ciclo de eventos (`app.exec_()`), no solo un `vtkRenderWindow` offscreen como el resto de la suite – automatizar esto de forma confiable (sin dejar procesos colgados) es trabajo adicional fuera del alcance de esta corrida. Es la única pieza de la secuencia de arranque real de `main.py` que falta medir.

- `filter_plugin._render_filtered` **no se benchmarkeó en vivo**, por la misma razón: construye un `QVTKRenderWindowInteractor` real y un `VTKContextMenu` que instancia un `QMenu` de Qt – sin una `QApplication` viva, PyQt5 puede abortar el proceso en vez de lanzar una excepción Python capturable. La cifra de §12 es una **estimación** derivada del micro-benchmark sintético, no una medición directa.
- `load_edf`/`load_mat` **sin cobertura** – no hay archivo de referencia en `test/data/`.
- **Los tiempos de renderizado VTK offscreen tienen variabilidad de proceso a proceso**, documentada en §10 con un ejemplo real (5.70 s vs. 725 ms para el mismo código). Tratar una única corrida como definitiva, especialmente para los benchmarks de render, es un error metodológico – de ahí la recomendación de repetir 2–3 veces en §16.
- **Todas las cifras son de una sola máquina** (§4). No se afirma que sean representativas de otro hardware – son una línea base válida *para comparar en esa misma máquina*, que es exactamente el uso previsto este mes.

18 Tabla completa de resultados (apéndice)

Todas las cifras de este informe, en un solo lugar, tal como quedaron guardadas en `test/performance_test/results/perf_results.json` bajo la etiqueta `gammalab1_baseline_2026-08-16`.

Benchmark	Mediana	n
`kernel.plugins.discover` (16 plugins)	19.6 ms	5
`kernel.plugin_manager.load_all` (16 plugins)	20.2 ms	5
`kernel.bootstrap_completo` (16 plugins)	19.9 ms	5
`kernel.register_plugin` (overhead, x200 ficticios)	1.4 ms	10
`fileio.load_abf` (2x1 800 000)	59.8 ms	5
`trials.cut_trials_single_channel` (30501x60)	18.6 ms	10
`filter.run_filter` (señal completa, 1 800 000)	43.2 ms	10
`filter.run_filter` (1 trial, 30501)	1.8 ms	20
`fft._compute_fft` (todos los trials)	3.8 ms	5
`fft_average._compute_fft_average`	11.5 ms	5
`psd._compute_psd` (Welch)	5.3 ms	5
`psd_average._compute_psd` (Welch)	9.2 ms	5
`relative_psd._compute_psd` (Welch)	6.3 ms	5
`relative_psd._compute_relative_psd`	0.07 ms	5
`wavelet.compute_wavelet` (1 trial)	215.6 ms	3
`wavelet_average.compute_wavelet` (x20 trials)	4190.2 ms	2
`wavelet.render_scalogram` (998x3051)	725.3 ms	3
`wavelet_average.render_scalogram` (998x3051)	741.6 ms	3
`erp._render_heatmap` (60x30501→60x1907)	35.8 ms	3
`adapters.dataset_to_vtk_table` (1800000x3)	21.0 ms	10
`adapters.trials_matrix_to_vtk_table` (30501x61)	11.1 ms	10
`vtk_array.bucle_python` (50 000 puntos)	4.6 ms	5
`vtk_array.numpy_to_vtk` (50 000 puntos)	0.031 ms	5
`vtk_array.bucle_python` (300 000 puntos)	28.4 ms	5
`vtk_array.numpy_to_vtk` (300 000 puntos)	0.432 ms	5
`data_store.add_signal` (x500)	13.0 ms	5
`data_store.set+get` (x2000 pares)	0.6 ms	5

19 Conclusión

Gamma Lab 1.0 es, para la enorme mayoría de sus operaciones (arranque, carga de archivos, FFT, PSD, `DataStore`), rápido: por debajo de 80 ms, imperceptible para el usuario. El rendimiento de la aplicación como un todo está, sin embargo, dominado por un número pequeño de puntos muy concentrados: el CWT de wavelet (un problema de threading, ya resuelto parcialmente en `wavelet_average` pero no en `wavelet`) y el renderizado VTK sin vectorizar de los mapas 2D y de al menos un gráfico de línea (`filter`) – un problema que, a diferencia del anterior, **ya tiene una solución probada y en uso dentro del mismo repositorio** (`core/utils/adapters.py`), solo que no se aplicó de forma consistente a los cinco lugares que comparten el mismo patrón.

Esta suite de benchmarks y este informe quedan pensados para no ser un documento de una sola vez: la sección 16 deja el procedimiento exacto para que, durante el desarrollo de Gamma Lab 2.0 este mes, cada decisión de arquitectura pueda contrastarse contra esta línea base con `compare_report.py`, en vez de depender de percepciones subjetivas de velocidad.